

Consider running a perceptron algorithm on a training set S arranged in a certain order. Now suppose if we run it with same initial weights and on the same training set but with a different order S' .

a) Does Perceptron make the same number of mistakes?

b) Does it end up with same final weights? If so prove it or give counterexample.

Solution: A perceptron update happens in the order that points are present (and only when they are misclassified). The final weights and number of updates depends on this order.

A Perceptron Convergence theorem states that if the data is linearly separable, the perceptron will eventually find a separating hyperplane after finitely many mistakes. The number of mistakes before converge is not fixed, it depends on the order of samples. The final separating hyperplane is not unique, there can be multiple valid separators, and perceptron may converge to different ones depending on the order.

a) Does Perceptron make same number of mistakes?

No, The mistake count depends on the order.

b) Does it end up with the same final weights?

No, If multiple separating hyperplane exists, the final weights differ depending on the update path.

Counter Example:

Positive examples: $(2,1)$ $(1,2)$

Negative examples: $(-2,-1)$ $(-1,-2)$

Both the vertical axis ($x=0$) & horizontal axis ($y=0$) separate these examples correctly.



If the perceptron sees given $(2,1)$ first, then the weight vector is pushed toward a vertical separator. (w aligned with x-axis)

If a

If a perceptron sees (1,2) first, then the updates push it towards a horizontal separator (w aligned with y-axis)
 so the final weights differ as the sequences differ.

Q2 Consider the following loss function which we denote $\phi(z) = \max(0, -z)$. let S be the training set $(x^1, y^1) \dots (x^m, y^m)$ where each $x^i \in \mathbb{R}^n$ and $y^i \in \{-1, +1\}$. Consider running a stochastic gradient descent to find a weight vector w that minimizes $\frac{1}{m} \sum_{i=1}^m \phi(y^i \cdot w^T x^i)$. Explain the explicit relationship between the algorithm and the Perceptron algorithm.

Solution: Analyzing the loss function $\phi(z)$
 If $z = y_i \bar{w}^T x_i > 0$, then $\phi(z) = 0$; Loss = 0, no update needed
 If $z \leq 0$, then $\phi(z) = -z$ which is $-(y_i \bar{w}^T x_i)$. Loss is positive, update is needed.

Finding Gradient of the loss function

$$\phi(z) = \phi(y_i \bar{w}^T x_i) = \begin{cases} 0, & \text{if } y_i \bar{w}^T x_i > 0 \\ -(y_i \bar{w}^T x_i), & \text{if } y_i \bar{w}^T x_i \leq 0 \end{cases}$$

In the case when $y_i \bar{w}^T x_i > 0$.

$$\frac{d}{dw} \phi(y_i \bar{w}^T x_i) = 0; \text{ as the gradient} = 0$$

In the case when $y_i \bar{w}^T x_i \leq 0$, we need to calculate

$$\frac{d}{dw_j} \phi(-y_i \bar{w}^T x_i) = -y_i x_{ij}$$

Partial derivative with respect to each weight w_j

Recall the SGD update rule happens on a random sample

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{d\phi(y; w^T x_i)}{dw}$$

if $y_i w^T x_i > 0$, then $w_{\text{new}} = w_{\text{old}}$; as $\frac{d\phi}{dw} = 0$

if $y_i w^T x_i < 0$, then $w_{\text{new}} = w_{\text{old}} - \eta (-y_i x_i)$
 $= \underline{w_{\text{old}} + \eta y_i x_i}$

The perceptron update step when misclassified

$$\underline{w_{\text{new}} = w_{\text{old}} + y_i x_i}$$

So they are both are identical, except SGD has a learning rate η , while perceptron uses $\eta=1$

Q3 let the domain be the Real line $\mathbb{R}$, and let $\mathcal{C}$ refer to the concept class of "3-piece classifiers", which are functions of form: for $\theta_1 < \theta_2$ and $b \in \{-1, +1\}$, $h_{\theta_1, \theta_2, b}(x)$ is b if $x \in [\theta_1, \theta_2]$ and $-b$ otherwise. In other words let $\mathcal{H}$ be the simple classifier of "decision stumps" i.e. functions $h_{\theta, b}$ such that $h(x)$ is b for all $x \leq \theta$ and $-b$ otherwise

~~Q3~~ (a) show formally that for any distribution on $\mathbb{R}$ and any unknown labeling function $c \in \mathcal{C}$ that is a 3-piece classifier, there exists a decision stump $h \in \mathcal{H}$ that has error at most $1/3$, i.e. $P(h(x) \neq c(x)) \leq 1/3$

Solution of (a):

A 3-piece classifier splits the real line into three regions: the interval and the two outside parts. Since the probabilities of these regions must add up to 1, at least one of them has probability of at most $1/3$. A decision stump can match labeling on two of the regions, but because it only makes one split, it cannot match all three. This means the stump will disagree with the target only on the remaining region, which is the smallest one. Since that region has at most probability of $1/3$, the stump error is bounded by $1/3$.

3. b) Describe a simple, efficient procedure for finding a decision stump that minimizes error with respect to a finite training set of size m . Such a procedure is called an empirical risk minimizer (ERM)

Solution To find the best stump, we can proceed as follows:

1. Sort the training examples by their x -values. This costs $O(m \log m)$ time.

2. Consider thresholds between each consecutive pair of sorted examples. There are at most $m-1$ such thresholds, and they represent all possible distinct stumps.

3. For each candidate threshold θ :

- Compute the training error if we assign $b = +1$ for $x \leq \theta$ and -1 otherwise.
- Compute the training error if we assign $b = -1$ for $x \leq \theta$ and $+1$ otherwise.

4. keep track of the threshold and direction that give the smallest number of mistakes

At the end of this process, we output the stump (θ, b) with the lowest training error. The algorithm runs in $O(m \log m)$ time and guarantees we have found the empirical risk minimizer.

3(c) Give a short intuitive explanation for why we should expect that we can easily pick m sufficiently large that the training error is a good approximation of the true error. i.e. why we can ensure generalization.

Solution: Decision stumps form a much simpler hypothesis class than 3-piece classifiers. Because the complexity of H is very low (its VC dimension is small), we know from learning theory that the training error of a stump will quickly converge to its true error as the sample size m increases. Intuitively, a simple classifier cannot overfit the data, the performance indicator on the training set is a good measure for the whole distribution. Thus with a sufficiently large training set, the best stump we find will generalize well.

By moving from learning the complex class C to weak learning with the simpler class H , we gain the ability to guarantee generalization.

Q4 Consider an iteration of the AdaBoost algorithm where we have obtained classifier h_t . Show that with respect to the distribution D_{t+1} generated for the next iteration, h_t has accuracy exactly $\frac{1}{2}$.

Solution:

$$\text{Error } E = \frac{1}{2} - \gamma, \quad \gamma > 0$$

$$\text{Accuracy } A = 1 - E = \frac{1}{2} + \gamma$$

$$\text{Update factor } \beta = \frac{E}{A} \Rightarrow \frac{\frac{1}{2} - \gamma}{\frac{1}{2} + \gamma}$$

Weight update rule with β

If $h_t(x_i) = y_i$; then multiply weight by β

If $h_t(x_i) \neq y_i$; then multiply weight by 1

Then normalize so weights sum = 1

After normalization, find the probability of $h_t(x) = y$ for the $t+1$ Distribution

$$P_{D_{t+1}}[h_t(x) = y] = \frac{A\beta}{A\beta + E}$$

$A\beta$ gives the total weight on the correct samples.

E gives weight on wrong samples

Plug in $\beta = \frac{E}{A}$

$$P_{D_{t+1}}[h_t(x) = y] = \frac{A \cdot (E/A)}{A \cdot (E/A) + E} = \frac{E}{E + E} = \underline{\underline{\frac{1}{2}}}$$

~~Q4~~